

Spindle: Efficient Distributed Training of Multi-Task Large Models via Wavefront Scheduling

Yujie Wang*
Peking University
Beijing, China
alfredwang@pku.edu.cn

Shenhan Zhu*
Peking University
Beijing, China
shenhan.zhu@pku.edu.cn

Fangcheng Fu*
Peking University
Beijing, China
ccchengff@pku.edu.cn

Xupeng Miao
Purdue University
West Lafayette, IN, USA
xupeng@purdue.edu

Jie Zhang
Alibaba Group
Beijing, China
wanglin.zj@alibaba-inc.com

Juan Zhu
Alibaba Group
Beijing, China
zhujuan.zj@alibaba-inc.com

Fan Hong
Alibaba Group
Beijing, China
hongfan.hf@alibaba-inc.com

Yong Li
Alibaba Group
Beijing, China
jiufeng.ly@alibaba-inc.com

Bin Cui*[†]
Peking University
Beijing, China
bin.cui@pku.edu.cn

Abstract

Recent foundation models are capable of handling multiple tasks and multiple data modalities with the unified base model structure and several specialized model components. However, efficient training of such multi-task (MT) multi-modal (MM) models poses significant system challenges due to the sophisticated model architecture and the heterogeneous workloads of different tasks and modalities.

In this paper, we propose Spindle, a brand new training system tailored for resource-efficient and high-performance training of MT MM models via wavefront scheduling. The key idea of Spindle is to decompose the model execution into waves and address the joint optimization problem sequentially, including both heterogeneity-aware workload parallelization and dependency-driven execution scheduling. We build our system and evaluate it on various MT MM models. Experiments demonstrate the superior performance and efficiency of Spindle, with speedup ratio up to 71% compared to state-of-the-art training systems.

*School of Computer Science & Key Lab of High Confidence Software Technologies (MOE), Peking University

[†]Institute of Computational Social Science, Peking University (Qingdao)

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. ASPLOS '25, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1079-7/2025/03

<https://doi.org/10.1145/3676641.3715992>

CCS Concepts: • Computer systems organization → Cloud computing; • Computing methodologies → Artificial intelligence; Parallel computing methodologies.

Keywords: Multi-Task Large Models; Distributed Training; Workload Heterogeneity

ACM Reference Format:

Yujie Wang, Shenhan Zhu, Fangcheng Fu, Xupeng Miao, Jie Zhang, Juan Zhu, Fan Hong, Yong Li, and Bin Cui. 2025. Spindle: Efficient Distributed Training of Multi-Task Large Models via Wavefront Scheduling. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*, March 30-April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3676641.3715992>

1 Introduction

In recent years, the field of artificial intelligence (AI) has witnessed a paradigm shift with the advent of large-scale foundation models [3, 11, 17, 58–60, 67, 68]. These models are equipped with extensive intrinsic knowledge, enabling them to be increasingly applied to a broad spectrum of downstream applications, including both the language domain (e.g., ChatGPT [51]) and many other data modalities (e.g., images [10, 13, 18, 21, 56], speech [7, 57, 70], video [6, 66]). The recent extension further involves composite scenarios [4, 5, 8, 9, 40, 46, 63, 73], where models are capable of processing and interpreting data across several tasks simultaneously. However, training these models is highly resource-intensive, requiring substantial GPU computing power. For example, Meta has announced to release the world-leading multi-modal model, LLaMA-3 [1], which has over 400B parameters and is trained on more than 48,000 GPUs.

Existing large model training systems are mainly designed for a single model with only one input data modality. Despite the extensive research and engineering efforts aimed at optimizing these systems from multiple perspectives, including distributed communication [50, 62, 71, 72], memory management [12, 61, 64], and GPU computation [15, 16], their performance is still limited when it comes to handling the increasingly complex requirements of multi-task (MT) multi-modal (MM) models. We identify two unique obstacles when building training systems for MT MM models.

One is the *workload heterogeneity* between different modalities or tasks. On the one hand, MM models often handle data that vary significantly in structure and size, demanding specialized preprocessing and computational approaches. For example, language models (e.g., GPT-family [3, 11, 58, 59], LLaMA-family [67, 68]) are usually equipped with dozens of layers with the same configuration (e.g., hidden size), while vision models may involve uneven layers to compute in various resolutions [39]. On the other hand, multiple tasks may leverage distinct data flows and activate individual model components, leading to inter-task workload heterogeneity. Existing training systems usually overlook such heterogeneity and apply sub-optimal training methodologies.

Another is the *execution dependency* among different model components. Recent MT MM model development usually adopts a sub-model sharing approach [8, 9, 22, 46, 73], where partial model layers containing common knowledge are shared across different modalities and tasks. As shown in Fig. 1, each data type also has its own learning component. Within every training iteration, the input data mixed with multiple modalities is simultaneously fed into the sophisticated model, where different model components are intricately activated and updated. To avoid redundant resource usage, the shared components are usually responsible for the data flows from multiple sources, resulting in execution barriers and blocking the following model layers. In addition, the proportion of different data modalities in MT workloads may shift over time due to task addition and completion, introducing further training complexity. To the best of our knowledge, none of existing training systems can deal with these unforeseen dependency efficiently due to the lack of understanding MT MM model execution.

To address these challenges, this paper introduces Spindle, a resource-efficient and high-performance training system for large-scale MT MM models. Considering the workload heterogeneity and execution dependency, a naïve solution is to *decouple* the model structure based on modality and task, replicate the shared components, and deploy them on separate devices. In this way, each sub-model can be optimized by existing systems, but it also brings significant *resource wastage and underutilization*, as well as additional overheads from replica synchronization. As an example, Fig. 1 showcases that such a naïve, decoupled execution suffers from fluctuating device utilization both intra-task and inter-task

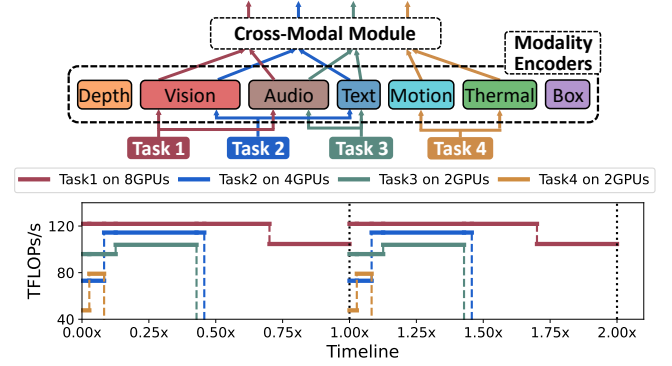


Figure 1. The upper portion illustrates the general model structure and training flow of MT MM training. The lower portion displays the current device utilization, measured in FLOPs per second, during the decoupled execution of four tasks across 2 iterations. Utilization fluctuation of different-colored and same-colored lines indicate inter-task and intra-task workload heterogeneity, respectively.

due to workload heterogeneity, leading to low or even idle GPU utilization for some time slots.

Instead of decoupling, Spindle manages to directly train the whole complex model *without* disjoint sub-model to minimize the resource usage. A key insight behind Spindle’s design is that *heterogeneous* and *dependent* sub-models can be decomposed into several sequentially executed *waves*. Specifically, Spindle treats a *wave* as the smallest scheduling unit for execution. Within each wave, the runtime engine concurrently executes multiple sliced *MetaOps* (i.e., continuously identical operators, as defined in §3.1). These sliced MetaOps are distributed across distinct and fixed groups of devices, while ensuring that their execution time costs are balanced. The concept of wave is central to Spindle’s design and operation. A more comprehensive definition of wave will be provided in §3.4, along with a concrete example of 6 waves illustrated in Fig. 5b.

To achieve resource-efficient and high-performance training of MT MM models, there are three key challenges for Spindle to address. In the following, we will introduce each challenge and how Spindle solves them.

First, finding the optimal model parallel configuration for heterogeneous workloads with diverse computational characteristics is a complex combinatorial problem. Existing single-model automatic parallelization approaches (e.g., Alpha [87], Unity [69], Galvatron [44, 75]) assume a spatial pipeline stage partition, and each operator (Op) is executed by all devices of the corresponding pipeline stage. Unfortunately, such assumptions only work for homogeneous models, failing to adapt to heterogeneous MT MM models.

Instead of solving the parallel configuration directly, Spindle captures the workload heterogeneity at the operator granularity and estimates its execution overheads under different

amount of allocated resources and parallel configurations (§3.2). The final configuration decision is left to the later step since it requires to be jointly optimized with considering the execution dependency. Spindle also introduces MetaOp to contract the graph (i.e., fusing continuous identical operators) to avoid redundant estimation overheads and shrink the problem scale (§3.1).

Second, breaking down the whole model into sequentially executed waves may easily result in inefficiencies. Determining the optimal division of waves is complicated since the operators differ significantly in execution overheads and have intricate operator dependencies.

Spindle addresses this problem with two steps: 1) Spindle’s *resource allocator* (§3.3) traverses the computation graph following the dependency topology and decides the optimal resource allocation for MetaOps in each candidate set (i.e., currently executable MetaOps). Here we reformulate this issue as a malleable project scheduling problem (MPSP) and subsequently derive the optimal solution. 2) After obtaining the parallel configuration of each MetaOp, the *wavefront scheduler* (§3.4) greedily slices and selects MetaOps to craft compact waves and minimizes the overall execution time.

Third, given the resource allocation plan and wavefront execution schedule, how to map them into physical devices is still a problem, as different mapping may lead to distinct inter-wave communication overheads and per-device memory consumption. To further improve the system efficiency, Spindle carefully considers these trade-offs and the real environment constraints (e.g., inter-device bandwidth, memory capacity) when generating the device placement plan (§3.5).

Our contributions are summarized as follows:

- We present Spindle, a resource-efficient and high-performance training system for MT MM models.
- We propose a jointly optimization framework to achieve heterogeneity-aware workload parallelization and dependency-driven execution scheduling.
- We build a general runtime engine to perform the wavefront schedule, automatically resolving execution dependencies at the wave boundaries.
- We evaluate Spindle on various MT MM models, and the results demonstrate the superior performance and efficiency of Spindle compared with the state-of-the-art baselines, with the speedup ratio up to 71%.

2 Preliminary

2.1 Multi-Task Multi-Modal Models

Foundation models, such as GPT series [3, 11, 59], LLaMA series [67, 68], have set new benchmarks across various language tasks and revolutionized deep learning. They’ve also been successfully adapted for other modalities and tasks, including image processing [10, 18, 39], audio processing [7, 57, 70], video analysis [6, 66]. Multi-modal models [8, 9, 14, 22,

56, 73] leverage these foundation models to integrate information from multiple data modalities. Multi-modal models typically have the multi-tower structure, utilizing multiple modality encoders to extract modality features, and a cross-modal module for feature alignment and fusing. Some of these models fuse modality information via lightweight contrastive learning objectives [22, 25, 29, 56, 80, 81, 84], with CLIP [56] being a notable example, and ImageBind [22] further extending CLIP to six modalities. Others fuse modalities via the language model with generative loss [19, 33, 34, 38, 73, 74, 83, 88], some leveraging the powerful pretrained LLMs.

Recently, researchers have begun to construct more complicated multi-task multi-modal (MT MM) models [4, 8, 9, 46, 55], enabling processing diverse multi-modal tasks within a unified model. This is because each modality encompasses various tasks, and each task often involves multiple modalities as well, and this reflects researchers’ aspiration towards general-purpose AI. Fig. 1 (upper side) illustrates the general structure and training flow of MT MM models. Flamingo [4] is among the first to handle multiple vision-language tasks. OFASys [9] proposes a general MT MM learning paradigm, as shown in Fig. 1, designing distinct modality encoders and cross-modal modules for different tasks and modalities, allowing the activation of different components as required by the task and modality at hand. For example, speech recognition and image captioning tasks shall activate and share the text encoder but feed the visual- and audio-inputs into different encoders. Many empirical results [4, 8, 9, 40, 55, 63, 73] have also shown that such a joint multi-task training paradigm achieves better multi-modal capabilities for MT MM models than performing single-task training separately.

2.2 Parallelisms in Distributed Training

As model sizes and training data volumes grow, modern DL systems commonly employ various parallelism techniques for distributed training on GPU clusters. Data parallelism (DP) [36, 61, 86] splits the input data, with each device handling a portion of the data storage and computation, and synchronizing model gradients across devices. Model parallelism [24, 28, 47, 48, 50] partitions model parameters, with each device responsible for a segment of the model. Model parallelisms can be categorized into two popular types: tensor parallelism (TP) partitions the model vertically [50], while pipeline parallelism (PP) [28, 47, 48] splits the model horizontally, organizing model execution into a pipeline. Contemporary distributed training systems, such as Megatron-LM [50] and DeepSpeed [62], leverage multiple parallelisms and implement a hybrid parallelism approach for model training. For example, Megatron-LM introduces 3D parallelism, which concurrently utilizes DP, TP, and PP. Researchers have also developed advanced automatic parallelism [31, 44, 75, 87] techniques to facilitate the tuning of optimal parallelism combinations, which integrates multiple parallelism dimensions, employ sophisticated optimization

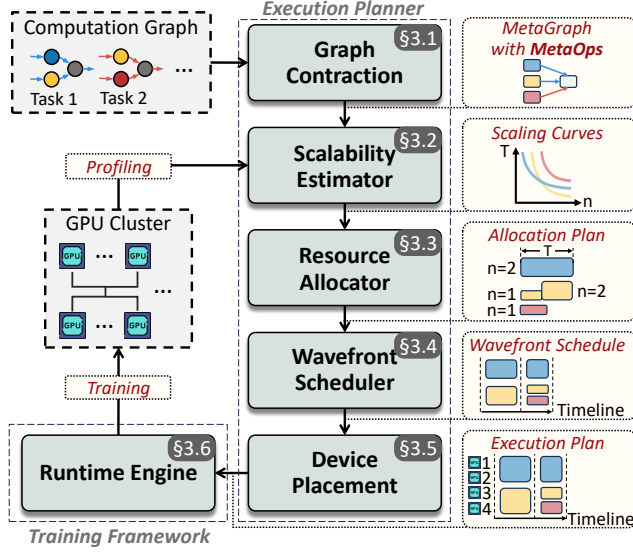


Figure 2. Architecture overview of Spindle.

workflows, and automatically determine the most efficient hybrid parallelism strategy. However, these existing training systems are mainly designed for single task and single model training, with limited performance on the complex scenario of training MT MM models.

3 System Design

Spindle is a highly efficient and scalable training framework for MT MM models. Fig. 2 depicts its system architecture, comprising the execution planner and the training framework. Given the diverse user-defined training tasks and the GPU cluster, the goal of Spindle is to devise the most efficient execution plan to facilitate effective MT MM training.

Problem Formulation. We formalize the optimization problem of Spindle as follows. Firstly, Spindle interprets the input tasks as a unified directed acyclic computation graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $i \in \mathcal{V}$ represents a computational operator and each edge $\langle i, j \rangle \in \mathcal{E}$ denotes the data flow from operator i to j . Each task activates specific operators and parameters with unique data flows. For instance, a vision-related task activates a vision Transformer layer as an operator, with image features serving as the data flow. The left side of Fig. 3 displays an example of a computation graph. Then, given the computation graph $\mathcal{G}$ and the GPU cluster with N devices, Spindle aims to minimize the maximal operator completion time C . Specifically, we need to find an execution plan P , which assigns each operator $i \in \mathcal{V}$ with an AS-tuple $\langle n_i, s_i \rangle \in \mathcal{U}$, such that the operator i is Allocated n_i devices and is Scheduled to execute from time s_i . Here the set $\mathcal{U} = \{\langle n, s \rangle | n \in \mathbb{N}, s \geq 0\}$ is formed by all valid AS-tuples. We further denote the execution time of operator i when allocated n_i devices as $t_i = T_i(n_i)$. Then, the

optimization problem is formulated as follows. Here (2) is the allocation capacity constraint for any time t , and (3) is the operator dependency constraint.

$$\arg \min_{P=\{i \rightarrow \langle n_i, s_i \rangle\} | i \in \mathcal{V}, \langle n_i, s_i \rangle \in \mathcal{U}} C := \max_{i \in \mathcal{V}} \{s_i + t_i\} \quad (1)$$

$$\text{s.t.} \quad \sum_{t \in (s_i, s_i + t_i), i \in \mathcal{V}} n_i \leq N \quad \text{for } \forall t \in \mathbb{R}^+ \quad (2)$$

$$s_i + t_i \leq s_j \quad \text{for } \forall \langle i, j \rangle \in \mathcal{E} \quad (3)$$

Sketch of Solution. Before stepping into the solution, we'd like to first present an overview for better readability. First, Spindle initiates a graph contraction process (§3.1), contracting the original graph $\mathcal{G}$ into a MetaGraph $\mathcal{G}_M$ composed of MetaOps (Fig. 3), where each MetaOp characterizes a unique workload. This process further decouples MetaOps into different MetaLevels, ensuring that there are no dependencies among MetaOps within the same MetaLevel. Second, the scalability estimator (§3.2) estimates the execution time and resource scalability for each MetaOp, producing scaling curves (Fig. 4). Following this, the resource allocator (§3.3) deduces the allocation plan for each MetaLevel individually (Fig. 5a). Given the allocation plan, the wavefront scheduler (§3.4) slices the MetaOps and organizes them into waves, and produces the wavefront schedule for execution. Subsequently, device placement (§3.5) strategies are then employed to assign MetaOps to appropriate devices, resulting in the Spindle execution plan (Fig. 5b). Finally, the runtime engine (§3.6) utilizes this plan to instantiate the model on each device and facilitate an efficient MT MM training process.

3.1 Graph Contraction

Depicting Workload Heterogeneity with MetaOps. Spindle minimizes the execution time by optimizing resource allocation and scheduling for each operator within $\mathcal{G}$. This optimization process necessitates an understanding of the workload characteristics for each operator $i \in \mathcal{V}$, which can be reflected by its execution time function $t_i = T_i(n_i)$, which varies with the device allocation amount n_i . Given that $\mathcal{G}$ typically includes a large number of operators while many of them share similar workload characteristics (such as stacked Transformer layers), Spindle initiates a graph contraction process to streamline the complicated graph. It categorizes operators based on their computational workload characteristics, as illustrated in Fig. 3. In this process, operators are contracted into a MetaOp if they meet the following criteria:

- (1) There is a data flow between operator i and j , i.e., $\langle i, j \rangle \in \mathcal{E}$, and both the out-degree of operator i and the in-degree of operator j are 1, ensuring that they are direct predecessors and successors to each other.
- (2) Operator i and j share the same operator type and input data size, confirming identical workloads.

During graph contraction, we traverse the original graph $\mathcal{G}$ in topological order, contracting operators based on the

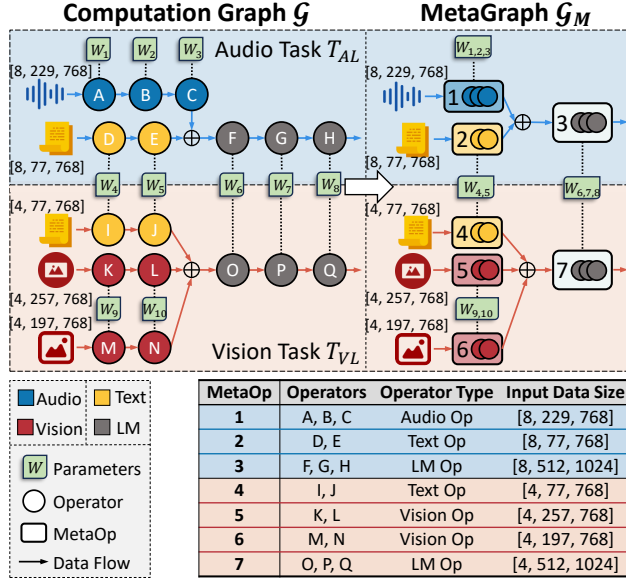


Figure 3. Computation graph $\mathcal{G}$ and contracted MetaGraph $\mathcal{G}_M$.

specified criteria until no further pairs of operators meeting these conditions. This results in a contracted MetaGraph $\mathcal{G}_M = (\mathcal{V}_M, \mathcal{E}_M)$, with each node $m \in \mathcal{V}_M$ representing a MetaOp that consists of L_m consecutive operators in $\mathcal{G}$. Since operators in the same MetaOp share the same workload, we slightly abuse the notation and denote the execution time function for each operator in MetaOp m as $T_m(n)$.

Disentangling MetaOp Dependency with MetaLevels.

To facilitate operator-level resource allocation and scheduling, we further introduce an abstraction called MetaLevel, which signifies the level of dependency. MetaOps at the same level are independent to each other. The level of each MetaOp can be derived by a Breadth-First-Search (BFS), with the level assigned based on the search depth, which inherently ensures no dependency among the MetaOps of same level. By doing so, the problem (1) can be dissected into several simplified sub-problems for different MetaLevels. Next, we introduce how Spindle derives the allocation and scheduling for each MetaLevel individually, and merges them into the final plan.

3.2 Scalability Estimator

As MetaOps differ in operator types and/or input data sizes, they characterize heterogeneous workloads and thus necessitate different amount of resources. Furthermore, these MetaOps have distinct resource scalability (i.e., how its execution time varies w.r.t. the amount of allocated resources). For instance, the left side of Fig. 4 shows the execution time of different MetaOps, $T_m(n)$, in Multitask-CLIP (a multi-task extension of CLIP, detailed in §5.1). Some MetaOps show almost linear decreases in execution time as resources increase

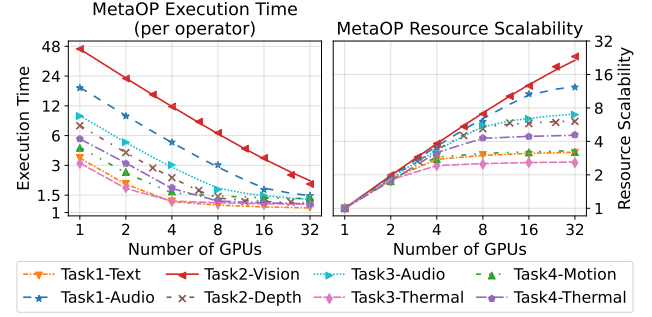


Figure 4. An example of the execution time and resource scalability of MetaOps in 4-task Multitask-CLIP, denoted as *scaling curves*.

(e.g., Task2-Vision), while others decrease much more slowly (e.g., Task1-Text). The right side of Fig. 4 further shows the value of $\zeta_m(n) = T_m(1)/T_m(n)$, which measures how much the operator accelerates when using more GPUs, and a value of $\zeta_m(n)$ closer to n signifies better resource scalability. As can be seen, different MetaOps not only have varying execution time, but also exhibit different resource scalability, posing a significant challenge for resource allocation.

In response to this issue, Spindle employs a scalability estimator to accurately capture the execution time and the resource scalability of each MetaOp. Previous works [44, 69, 87] have designed effective estimation methods for distributed training, commonly utilizing the α - β modelling [26]. However, although this may work well for homogeneous workloads (e.g., LLMs with homogeneous layers), we find that it does not fit the workload heterogeneous nature of MT MM models. This is because different MetaOps have distinct workload and resource scalability, and the invoked kernels may vary across different per-device workloads, therefore causing distinct performance. In a nutshell, our scalability estimator adopts the *piecewise* α - β modelling for more accurate estimation of heterogeneous MT MM workloads. Given the target MT MM model, it profiles several discrete data points $(n_i, T_m(n_i))$ for each MetaOp under different parallel configurations, and then fits the curve of piecewise α - β function. To estimate the execution time $T_m(n)$, it locates the range that n falls into, and returns the estimated time according to the corresponding piecewise function. In practice, the profiling and estimating process for each MT MM model takes within 5 minutes, which is negligible compared to the training time. In Fig. 4, the scatter points represent empirical measurements, while the curves depict the function estimated by our scalability estimator, which we denote as *scaling curves*. As can be seen, our scalability estimator effectively and accurately estimates the execution time $T_m(n)$ for each MetaOp. More details are illustrated in Appendix A [2].

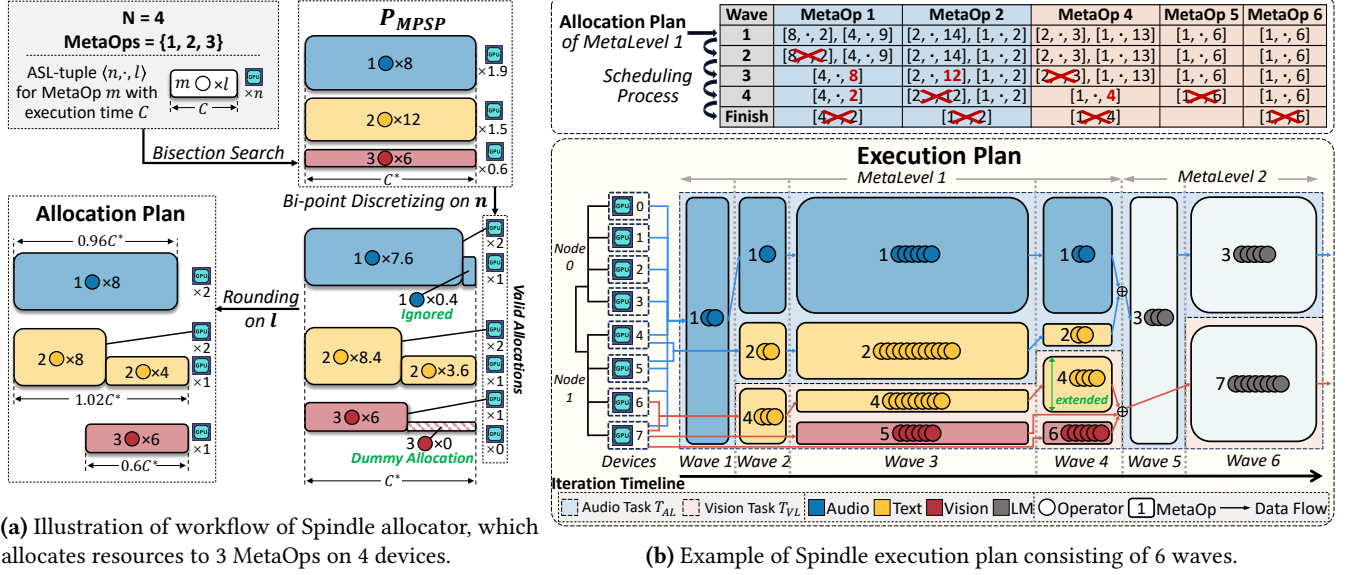


Figure 5. Illustration of Spindle allocator and Spindle execution plan.

3.3 Resource Allocator

We now introduce our resource allocator, which allocates appropriate computational resources to MetaOps. We first transition problem (1) into the sub-problem on MetaLevel. We then detail our allocation strategies, which first relax constraints and optimize the continuous problem, and then discretize the optimal solution for practical allocation plans.

Problem Formulation on MetaLevel. We first re-formulate the problem (1) on one MetaLevel with a set of MetaOps denoted by $\tilde{\mathcal{V}}_M$. In this formulation, we split each MetaOp into different execution part, by assigning it with several ASL-tuples $\langle n, s, l \rangle \in \mathcal{U}_M$, such that l consecutive operators of this MetaOp are scheduled to execute from time s with n devices. Here $\mathcal{U}_M = \{\langle n, s, l \rangle | n, l \in \mathbb{N}, s \geq 0\}$ is formed by all valid ASL-tuples. For each MetaOp $m \in \tilde{\mathcal{V}}_M$, its execution plan is a set of ASL-tuples P_m . For a MetaLevel, the execution plan P consists of P_m for all MetaOps $m \in \tilde{\mathcal{V}}_M$, i.e., $P = \{m \rightarrow P_m\}$. Given $m \in \tilde{\mathcal{V}}_M$ and one ASL-tuple $p = \langle n_m^{(p)}, s_m^{(p)}, l_m^{(p)} \rangle \in P_m$, we denote the execution time span, end time, and time interval by $t_m^{(p)} = T_m(n_m^{(p)}) \cdot l_m^{(p)}$, $e_m^{(p)} = s_m^{(p)} + t_m^{(p)}$, and $I_m^{(p)} = (s_m^{(p)}, e_m^{(p)})$, respectively. The problem can be re-written as:

$$\arg \min_{P=\{m \rightarrow P_m | m \in \tilde{\mathcal{V}}_M, P_m \subset 2^{\mathcal{U}_M}\}} \tilde{C} := \max_{m \in \tilde{\mathcal{V}}_M, p \in P_m} \{e_m^{(p)}\} \quad (4)$$

$$\text{s.t.} \quad \sum_{t \in I_m^{(p)}, m \in \tilde{\mathcal{V}}_M, p \in P_m} n_m^{(p)} \leq N \quad \text{for } \forall t \in \mathbb{R}^+ \quad (5)$$

$$I_m^{(p_1)} \cap I_m^{(p_2)} = \emptyset \quad \text{for } \forall m \in \tilde{\mathcal{V}}_M, p_1, p_2 \in P_m \quad (6)$$

$$\sum_{p \in P_m} l_m^{(p)} = L_m \quad \text{for } \forall m \in \tilde{\mathcal{V}}_M \quad (7)$$

Compared with the original problem (1), the sub-problem (4) on MetaLevel gets rid of the dependency constraint, while the constraint (6) enforces the execution intervals of ASL-tuples in P_m to be pairwise disjoint, because operators within the same MetaOp cannot execute simultaneously, and (7) ensures all operators are executed for each MetaOp.

Optimum of the Continuous Problem. If we relax the constraints, allowing GPU resources and operators to be continuously divisible (i.e., n and l in ASL-tuples are not limited to integers), the problem is transformed into a well-established problem, malleable project scheduling problem (MPSP), with malleable projects and continuously divisible resources [20]. We denote the optimal solution of this relaxed problem by P_{MPSP} . Prior works [76, 77] have given the following theorem.

Theorem 1. *If the execution time functions $T_m(n)$, $n \in \mathbb{R}^+$, are positive and non-increasing for every MetaOp $m \in \tilde{\mathcal{V}}_M$, then $P_{MPSP} = \{m \rightarrow P_m\}$ satisfies that $P_m = \{\langle n_m^*, 0, L_m \rangle\}$, $\forall m \in \tilde{\mathcal{V}}_M$, where the optimum objective $\tilde{C}^*$ and allocations n_m^* can be found from*

$$T_m(n_m^*) \cdot L_m = \tilde{C}^* \text{ for } \forall m \in \tilde{\mathcal{V}}_M \text{ and } \sum_{m \in \tilde{\mathcal{V}}_M} n_m^* = N. \quad (8)$$

From Theorem 1, it follows that in the optimal situation, all MetaOps start simultaneously, execute all their operators, and finish together. They share an identical end time $e_m = \tilde{C}^*$, which is exactly the minimized operator completion time.

To achieve P_{MPSP} , our allocator utilizes the scaling curves from §3.2 to acquire an estimation of $T_m(n)$, and performs a bisection search procedure over $\tilde{C}^*$ with the following

equation. The details are illustrated in Appendix B [2].

$$\sum_{m \in \mathcal{V}_M} T_m^{-1} (\tilde{C}^* / L_m) = N. \quad (9)$$

Bi-point Discretized Allocation. From the continuous problem, we've determined the optimal time $\tilde{C}^*$, as well as the optimal allocations for each MetaOp, n_m^* , which is a real number. To reinstate n 's as integers, our allocator computes each MetaOp's proper discrete allocations individually. For every MetaOp m , it uses two discrete ASL-tuples $\langle \bar{n}_m, \cdot, \bar{l}_m \rangle, \langle \underline{n}_m, \cdot, \underline{l}_m \rangle$ to linearly represent the continuous, optimal solution $\langle n_m^*, 0, L_m \rangle$ in P_{MPSP} . To preserve the optimum property of P_{MPSP} , we require the discretized allocation plan to satisfy the following two conditions:

$$L_m = \bar{l}_m + \underline{l}_m \quad (10a) \quad \tilde{C}^* = T_m(\bar{n}_m) \cdot \bar{l}_m + T_m(\underline{n}_m) \cdot \underline{l}_m \quad (10b)$$

Cond. (10a) ensures these two discrete ASL-tuples complete the workload of MetaOp m , and Cond. (10b) ensures their total execution time is exactly equal to the minimum operator completion time $\tilde{C}^*$ in P_{MPSP} , thus perserving the optimum property. Here we first select $\bar{n}_m, \underline{n}_m$ as the closest *valid* integer numbers such that $n_m^* \in [\underline{n}_m, \bar{n}_m]$, and $\bar{l}_m, \underline{l}_m \in \mathbb{R}^+$ are derived naturally. For instance, as shown in Fig. 5a, MetaOp 2 with $n_2^* = 1.5, L_2 = 12$ in P_{MPSP} is discretized as $\bar{n}_2 = 2, \underline{n}_2 = 1$ and $\bar{l}_2 = 8.4, \underline{l}_2 = 3.6$ in this step. Here we impose the *valid* constraint on the allocation n for MetaOp m for practical reasons. For instance, if an MetaOp is applied data parallelism, its allocation n is supposed to divide its global batch size B_m to avoid resource under-utilization due to uneven partition of samples. For another example, if an MetaOp is applied tensor parallelism or sequence parallelism with degree 2, its allocation n is supposed to be divisible by this degree, thus $n = 3, 5, 7$ as invalid. Such *valid* constraint ensures the allocation plan for each MetaOp is practical. Specially, allocation with $\underline{n}_m = 0$ is treated as a dummy allocation (e.g., MetaOp 3 in Fig. 5a), which preserves the optimum property of Cond. (10b) but will then be ignored.

Then, we reinstates l 's as integers by rounding $\bar{l}_m, \underline{l}_m$ to the nearest integers. If the rounded l equals 0, this ASL-tuple will be ignored. This rounding procedure preserves the integrity of Cond. (10a) and introduces only minor bias to Cond. (10b). Finally, the discretized ASL-tuples of all MetaOps form the allocation plan. Note that the allocation plan only ensures the longest execution time among all MetaOps is approximately $\tilde{C}^*$, yet it does not specify the start time for each ASL-tuple, which is determined by wavefront scheduler in §3.4.

3.4 Wavefront Scheduler

Given the allocation plan from the resource allocator, we now describe how Spindle schedules the execution of MetaOps. We first introduce the concept of *wave*, the scheduling unit of Spindle. Then we present our wavefront scheduling, which

Algorithm 1: Wavefront Scheduling for one MetaLevel

Input: # Devices N , start time T_{start} ,
 $alloc_plan = \{m \rightarrow \{\langle \bar{n}_m, \cdot, \bar{l}_m \rangle, \langle \underline{n}_m, \cdot, \underline{l}_m \rangle\}\}$
Output: Wavefront schedule $P = \bigcup_k S_k$, end time T_{end}

```

1  $T_{current} \leftarrow T_{start}; P \leftarrow \emptyset; S_{remain} \leftarrow alloc\_plan;$ 
2 while  $S_{remain}$  is not empty do // schedule for wave  $k$ 
3    $S_{cand} \leftarrow Propose\_Candidate\_Set(N, S_{remain});$ 
4    $S_{cand} \leftarrow Extend\_Resources\_If\_Needed(S_{cand});$ 
5    $T_{wave}, S_{sched} \leftarrow Align\_Time\_Span(S_{cand});$ 
6    $S_k \leftarrow Set\_Start\_Time(S_{sched}, T_{current}); P \leftarrow P \cup S_k;$ 
7    $S_{remain} \leftarrow S_{remain} - S_{sched}; T_{current} \leftarrow T_{current} + T_{wave};$ 
8 return  $P, T_{current}$ 
```

schedules the execution of MetaOps greedily for each wave. Finally, the operator dependencies among MetaLevels are reinstated by merging the wavefront schedules together.

Definition of wave. It is worthy to note that, although Theorem 1 implies that all MetaOps share the same start and end time in the continuous form, this property does not hold after the discretization process. The reason is that the execution time of ASL-tuples may vary, or the resources are insufficient to execute all tuples concurrently. To cope with this problem, we devise a fine-grained wavefront scheduler that slices the MetaOps and selects a few of them to execute concurrently on different groups of devices. We define *wave* as the smallest scheduling unit, which corresponds to one concurrent execution as aforementioned. The wavefront scheduler attempts to minimize the device idle time in each wave, by (1) occupying the devices as many as possible to maximize device utilization (Wavefront Scheduling step ①), and (2) aligning the execution time spans of different sliced MetaOps to avoid idle time (Wavefront Scheduling step ③). As illustrated in Fig. 5b, resource (device) allocation remains unchanged in one wave, and transmission of data flow occurs only between two waves. Next, we introduce our greedy algorithm that crafts the waves to form the scheduling plan.

Wavefront Scheduling. As outlined in Alg. 1, the scheduler iteratively crafts waves in a greedy manner. Below we discuss how one wave is crafted with Fig. 5b as an example.

① First, the scheduler greedily proposes ASL-tuples to form a candidate set, aiming to utilize as many devices as possible (line 3). For instance with Fig. 5b, the scheduler proposes the first ASL-tuple of MetaOp 1 to craft wave 1 since it occupies all devices. Similarly, for wave 2, it proposes the ASL-tuples of MetaOp 1, 2, and 4, which correspond to 4, 2, 2 devices, respectively, in order to make full use of all devices.

② If the candidate set fails to occupy all devices, the cluster resources will be underutilized. To address this issue, we extend the allocated resources in specific tuples to ensure all devices are utilized (line 4). For instance, in wave 4 of Fig. 5b, the allocation of MetaOp 4 is extended from 1 device to 2

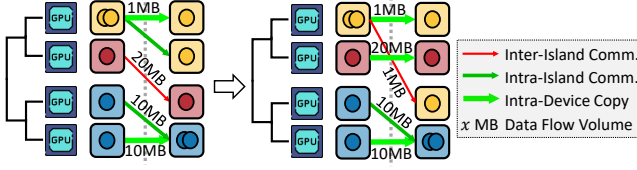


Figure 6. Illustration of Spindle device placement.

devices. Resource extension is prioritized for MetaOps with larger remaining execution time, with the hope of balancing the remaining workload among the MetaOps.

③ In most cases, the proposed ASL-tuples differ in execution time. If we directly craft a wave with them, it would be inefficient since there must be idle devices. Fortunately, this can be avoided by dissecting the ASL-tuples to align their time span (i.e., only a few number of operators in the MetaOp are scheduled in this wave). For instance, in wave 2 of Fig. 5b, the proposed ASL-tuples for MetaOp 1, 2, and 4 correspond to 9, 14, and 3 operators, respectively. To align the execution time, the ASL-tuples for MetaOp 1 and 2 are dissected, with only 1 and 2 operators of them being scheduled, while the remaining 8 and 13 operators left to be scheduled in subsequent waves. Our scheduler simply aligns the time span w.r.t. the ASL-tuple with shortest execution time (e.g., the one for MetaOp 4 in previous example), and computes the aligned time span as the duration of current wave (line 5).

④ After the time span alignment, the scheduler concludes the current wave (lines 6–7), including specifying the start time for operators that are scheduled in this wave, and removing them from the remaining set.

Merging MetaLevels. As stated in §3.1, MetaOps are decoupled into MetaLevels to disentangle operator dependencies. Spindle invokes the aforementioned allocation and scheduling for each MetaLevel individually, and merges their wavefront schedules together as the final execution schedule.

3.5 Device Placement

Given the wavefront schedule, which consists of the allocation amount and execution time of each MetaOp, we now discuss how Spindle determines the specific devices for each MetaOp, known as device placement, which affects the inter-wave communication overhead and memory consumption. Spindle employs several guidelines based on empirical insights or observations to optimize device placement.

Intra-Device-Island Placement. Placement within a device island is always preferred for each MetaOp and data flow between MetaOps. A device island consists of devices connected by high-bandwidth interconnects (e.g., NVLink), typically adjacent devices, such as adjacent GPUs within one node. For MetaOps, prioritizing intra-island placement reduces the potential intra-MetaOp communication costs. For data flow between MetaOps across waves, intra-island

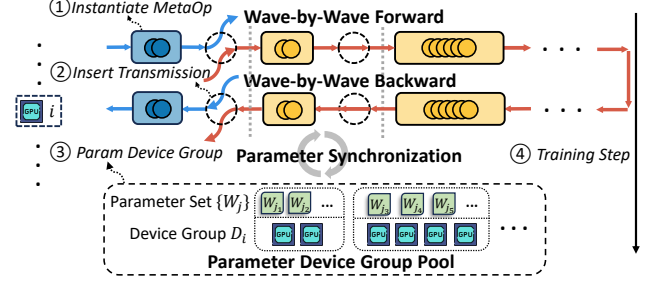


Figure 7. Illustration of Spindle runtime engine.

placement reduces transmission costs leveraging the high intra-island bandwidth or even faster intra-device copying.

Prioritizing High Communication Workloads. When it's infeasible to place all MetaOps and data flows within the device island, Spindle will estimate the communication volume of each MetaOp and data flow to prioritize placing those with higher volumes within a device island. For instance, in Fig. 6, the data flow volume between red MetaOps is significantly higher than that between yellow ones. Therefore, Spindle prefers to place the data flow between red ones within the island. This guideline ensures the most communication-intensive components receive the most efficient hardware configuration to minimize communication overhead.

Device Memory Balance. As each device holds heterogeneous MetaOps, the memory overhead varies across devices. Placing too many memory-intensive MetaOps on a single device may cause out-of-memory (OOM) errors. Therefore, Spindle actively balance the memory load across all devices during placement. Specifically, Spindle estimates each MetaOp's memory consumption, tracks available memory on devices, and prioritizes placement on the device with the most available memory. Besides, for MetaOps sharing the same parameters, we prioritize placing them on the same device to minimize redundant storage.

Based on these guidelines, Spindle performs device placement wave by wave greedily, prioritizing the minimization of communication overhead, such as inter-wave transmission, while simultaneously maintaining device memory balance. When OOM occurs due to imbalanced placement, Spindle will consider alternative placements with sub-optimal communication costs and better memory balance. If necessary, backtracking is employed to adjust the placements from earlier waves to effectively address the OOM issues.

3.6 Runtime Engine

The runtime engine is responsible for running the execution plan to facilitate efficient MT MM training, which is more complex than conventional single-task training, as each device handles heterogeneous MetaOps and local computation graphs. Spindle runtime engine operates in four main steps:

(1) **Localization.** Initially, Spindle localizes the execution plan to each device. Specifically, each device instantiates the corresponding MetaOp of each wave locally, and initializes the required model components and parameters.

(2) **Intra-task Data Dependency.** Secondly, Spindle inserts transmission operators to connect the MetaOps across waves to handle the data flow dependencies, including forward activations and backward gradients. According to the devices of MetaOps and data format requirements, operations such as *copy*, *shard*, *concat*, *send*, and *receive* are used to transmit data flows with minimal overhead. This step not only correctly handles data flow dependencies between MetaOps but also links the MetaOps on each device into a complete local computation graph ready for execution.

(3) **Inter-task Model Dependency.** Then, Spindle manages parameter device groups for synchronization among various tasks by maintaining a global parameter device group pool. During each iteration, for each parameter W_j , all tasks or modalities that activate it on different devices contribute to its gradient, which needs to be accumulated and synchronized to facilitate parameter sharing. Therefore, before the training process, Spindle scans all devices to determine the device group D_i for each parameter W_j , which represents W_j is shared and should be synchronized within group D_i . For efficiency, Spindle manages parameters with the same device group collectively and maintains a global parameter device group pool $\{D_i \rightarrow \{W_j\}\}$, where each device group D_i corresponds to a set of parameters $\{W_j\}$.

(4) **Training Step.** Finally, the training process is ready to begin. In each iteration of Spindle, each device executes the forward and backward propagation of the local computation graph in a wave-by-wave manner, which is comprised of the interleaved execution of MetaOps and transmission of data flow. Following the forward and backward phases, Spindle performs group-wise parameter synchronization to maintain the parameter consistency. Specifically, each parameter set $\{W_j\}$ is synchronized within its corresponding device group D_i in the parameter device group pool.

4 Implementation

Spindle is an efficient and scalable MT MM training system built on PyTorch with 10K Loc in Python: 2.1K LoC for the execution planner and 7.9K LoC for the runtime engine. We implement the data flow transmission with NCCL batched P2P primitives and the parameter device groups with NCCL communication groups. Spindle provides the users with simple, user-friendly and flexible API for defining MT MM training workloads. Specifically, training tasks in Spindle are represented as *SpindleTask*, and users can define various multi-modal tasks by customizing PyTorch modules and connecting them flexibly through the *add_flow* API in Spindle. Alternatively, user can also define different computational logic for various tasks implicitly within a single

Table 1. Experimental setups.

(a) Heterogeneity awareness of system competitors.

Competitors	Inter-Task	Intra-Task
Megatron-LM / DeepSpeed	✗	✗
DistMM-MT	✗	✓
Spindle-Optimus	✓	✗
Spindle	✓	✓

(b) Configuration of MT MM models for evaluation.

MT MM Model	Multitask-CLIP	OFASys	QWen-VAL
# Param.	1.20B	0.66B	9.25B
# Modalities	6	6	3
# Tasks	10	7	3
Cross-Modal Module	Contrastive Loss	Enc-Dec LLM	Dec-only LLM

unified model. Spindle can automatically split the modules and construct *SpindleTasks* via PyTorch FX Tracer, streamlining task definition. After the definition of multi-modal tasks, Spindle conducts the optimization workflow automatically, as illustrated in Fig. 2, and the Spindle runtime engine provides efficient and scalable model training process.

5 Experiments

5.1 Experimental Setups

Competitors. We compare Spindle with SOTA (state-of-the-art) distributed training systems, Megatron-LM [50] and DeepSpeed [62]. We also introduce other two systems that represent typical strategies for multi-task training, considering inter-task and intra-task heterogeneity respectively. Tab. 1a summarizes the features of competitors.

(1)&(2) **Megatron-LM & DeepSpeed:** Megatron-LM [50] and DeepSpeed [62] are widely used SOTA training systems tailored for single-task training. The naïve approach to train MT MM models on these systems is to decouple all sub-models on separate devices (§1), which requires plenty of resources and is impractical. Therefore, we decouple sub-models on temporal dimension within each iteration, where each sub-model takes up the whole cluster within a short time period, and is dependently and sequentially executed.

(3) **DistMM-MT:** DistMM [27] is a recent training system designed for multi-modal models, but focusing on single task only. DistMM-MT represents a multi-task (MT) extension of DistMM. It decouples multi-tasks, and for each single MM task allocates appropriate resources to different multi-tower modality encoders. Then it executes tasks sequentially.

(4) **Spindle-Optimus:** This baseline represents a workload-aware task-level resource allocation strategy, which adapts

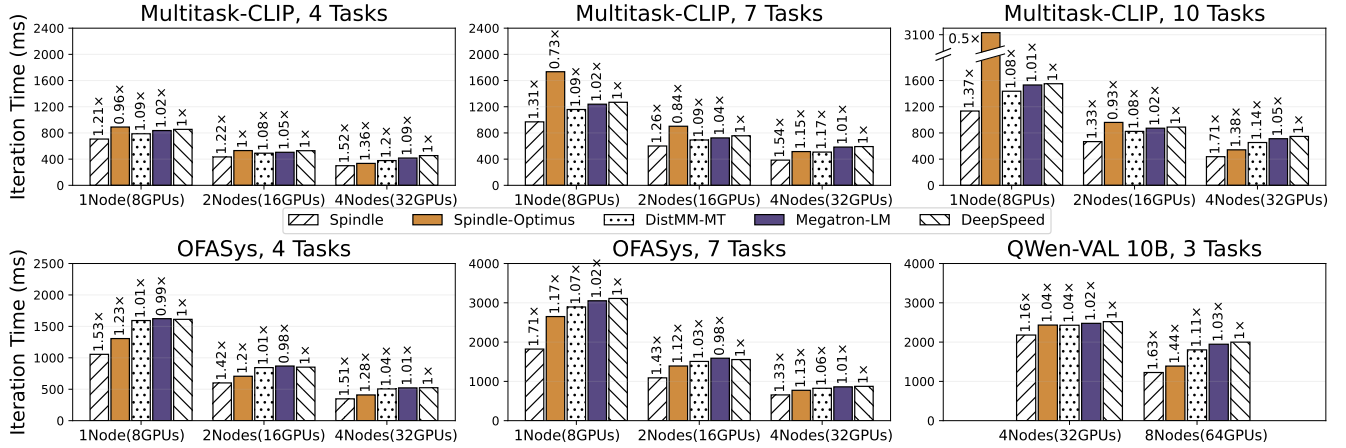


Figure 8. End-to-end performance comparison for Spindle and baseline systems. Shorter bars indicate superior system performance. The numbers above the bars denote each system’s speedup compared to DeepSpeed (larger than 1 is faster).

allocations according to the workload at the task level granularity. It’s inspired by Optimus [53], an effective cluster job scheduling system which proposes a greedy resource allocation scheme and iteratively assigns devices to the job that has the largest marginal gain. Despite differences between job scheduling and multi-task training (§6), we apply a similar principle and devise the marginal gain as $(T_m^{(c)}(n) - T_m^{(c)}(n')) / (n' - n)$, i.e., the task completion time reduction scaled by the allocation increment from n to n' . Here n' is the next valid allocation number larger than n .

Experimental Workloads. As illustrated in Tab. 1b, we select three models to represent popular MT MM workloads and conduct experiments on these workloads.

- (1) **Multitask-CLIP**, which adopts the same structure of ImageBind [22], is a multi-task variation to the classic and pioneer CLIP [56] model. Many multi-modal models [22, 25, 29, 56, 80, 84] follow this paradigm for multi-task training. Its cross-modal module (contrastive loss), has much smaller workload compared to its modality encoder, where most computation occurs.
- (2) **OFASys** [9] further generalizes the MT MM paradigm, using a unified LM of encoder-decoder structure for cross-modal processing. In OFASys, the cross-modal module’s workload is comparable to that of the modality encoders.
- (3) **QWen-VAL** [8, 14] adopts a modern, compute-intensive decoder-only LLM, with the workload of the cross-modal module usually larger than modality encoders. Recent multi-modal models like SPHINX-X [37], DeepSpeed-VisualChat [82], and BLIP-2 [33], employ this structure.

These workloads effectively represent the majority of MT MM workloads (and different workload distribution between modality encoders and cross-modal modules in Fig. 1), regardless of specific model structure variations.

Protocols. We conduct experiments on an 8-node GPU cluster. Each node consists of 8 NVIDIA A800 80 GB GPUs equipped with NVLink, and the nodes are interconnected by 400 Gbps InfiniBand. Since the baseline systems do not support automatic planning given a targeted MT MM model training workload, we manually tune their parallel configurations and memory optimization techniques (e.g., data and tensor parallelism degree, ZeRO stage, activation checkpointing, and etc.) to achieve the best performance. Averaged training time over 100 iterations is reported.

5.2 End-to-End Performance

Fig. 8 displays end-to-end iteration time comparisons between Spindle and baseline systems across various model workloads, multi-modal task configurations, and cluster sizes.

Comparison with SOTA systems. In general, compared to SOTA training systems, i.e., Megatron-LM and DeepSpeed, Spindle achieves speedup ratios of up to 67% and 71%, respectively. Below we delve into details.

To begin with, Spindle consistently outperforms the competitors across different task configurations. Notably, Spindle excels when handling a larger number of tasks. On the 10-task Multitask-CLIP and 7-task OFASys workloads, Spindle achieves speedup ratios ranging from 31% to 71% compared to SOTA systems. This underscores Spindle’s excellent scalability with increasing task counts.

In addition, Spindle consistently achieves optimal performance across various cluster sizes. On Multitask-CLIP, Spindle achieves the highest speedup ratios of 37%, 33%, and 71% on 8, 16, and 32 GPUs, respectively. Notably, Spindle maintains high efficiency even when the scalability of SOTA systems begins to diminish — that is, when the increase in resources does not correspond to significant speed improvements. For example, in 4-task Multitask-CLIP, expanding the cluster size from 16 to 32 GPUs results in only

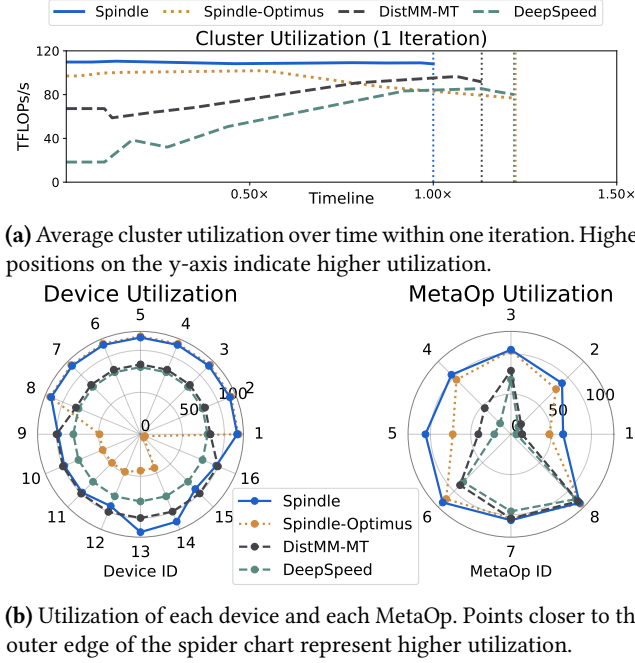


Figure 9. Case study of Multitask-CLIP (4 tasks, 16 GPUs). Utilization is measured in computation FLOPs per second.

modest speedup of up to $1.21\times$ for SOTA systems, whereas Spindle still achieves a $1.45\times$ speedup. This efficiency stems from Spindle’s heterogeneity-aware and operator-level fine-grained resource allocation and scheduling. For example, for a lightweight audio operator, DeepSpeed needs to parallelize it on the whole cluster with 16 GPUs due to its workload-unaware nature, causing the computational kernel to be underutilized or even idle, while Spindle may parallelize it with only 4 GPUs to ensure high utilization. Besides, when scaling from 16 to 32 GPUs, Spindle may maintain 4-GPU-allocation for the lightweight operator to keep high utilization.

More importantly, Spindle also exhibits excellent scalability w.r.t. model size. On larger model QWen-VAL, Spindle achieves a maximum speedup of $1.16\times$ on 32 GPUs and $1.63\times$ on 64 GPUs. Notably, when training QWen-VAL, Spindle achieves a $1.78\times$ speedup when scaling from 32 to 64 GPUs, whereas SOTA systems only achieve up to $1.27\times$ speedup. This is unsurprising since Spindle allows flexible allocations to avoid the unsatisfactory scalability of MetaOps with light workloads, as discussed above and in §3.2.

Comparison with other baselines. Next, we discuss the performance of other baselines, i.e., Spindle-Optimus with task-level resource allocation strategies, and DistMM-MT with the single-task strategy for MM models.

Spindle-Optimus, employing workload-aware task-level resource allocation, exhibits great performance, especially in larger-scale clusters, with the speedup ratio up to 44% compared to DeepSpeed. However, there are still many scenarios

where it underperforms. Its task-level strategy overlooks the workload heterogeneity within tasks, thereby limiting training efficiency. Moreover, the coarse granularity at task level can sometimes fail to achieve ideal load balancing among tasks, causing many devices to become idle during the latter part of the iteration. In comparison, Spindle enables finer-grained strategy, with operator-level resource allocation and wavefront load balancing, consistently achieving higher efficiency compared to Spindle-Optimus.

DistMM-MT also performs better than SOTA systems in most cases, with the speedup up to 20%, benefiting from its intra-task workload awareness and resource allocation. However, it’s designed for single-task MM models, which decouples tasks and optimizes each one separately, making it far from achieving the global optimum in multi-task cases. For OFASys, DistMM-MT shows poor performance. This is because DistMM-MT gains acceleration by parallelizing sub-models in the same task. However, OFASys uses a lightweight text adaptor, so most tasks that pair a modality with text are dominated by the other modality, making the sub-model parallelization ineffective. Compared to DistMM-MT, Spindle jointly optimizes the allocation and scheduling of all tasks and operators, therefore consistently outperforming the single-task strategy of DistMM-MT, achieving a speedup ratio of up to 59%.

5.3 Case Study

To better understand the advantages and performance gain of Spindle over the other competitors, we further conduct an in-depth case study of Multitask-CLIP (4 tasks, 16 GPUs). Fig. 9 presents system performance considering three key metrics: cluster average utilization over time, average utilization per device, and computational utilization of each MetaOp.

Firstly, DeepSpeed, representing SOTA systems, which executes the tasks sequentially with all resources, experiences fluctuating utilization due to the workload heterogeneity, leading to generally low overall utilization. Spindle-Optimus, which allocates resources at task level, improves cluster utilization at the iteration beginning, but as tasks with light workloads finish, more devices become idle, declining overall utilization. DistMM-MT manages to enhance utilization via intra-task resource allocation, but the ignorance of inter-task heterogeneity limits its utilization. In contrast, Spindle maintains consistently high utilization throughout the iteration.

Furthermore, Spindle significantly elevates the utilization of all devices and MetaOps, showcasing its superior handling of workload balance via operator-level strategies. In contrast, DeepSpeed shows lower utilization across all devices and MetaOps. Although task-level strategies of Spindle-Optimus can enhance the utilization of certain devices, the coarse granularity of allocation inevitably leads to workload imbalances, leaving many devices underutilized. DistMM-MT also improves the utilization of certain devices and MetaOps, but the results are still unsatisfactory as it fails to reach the

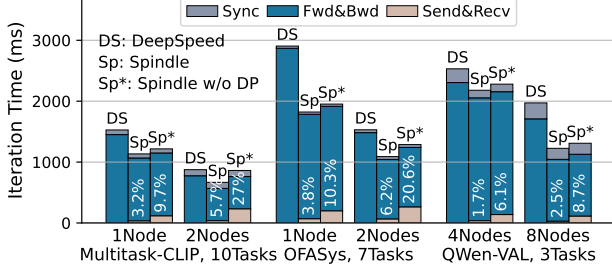


Figure 10. Time breakdown analysis. Percentage of inter-wave *send & receive* in total time is labeled for ablation study.

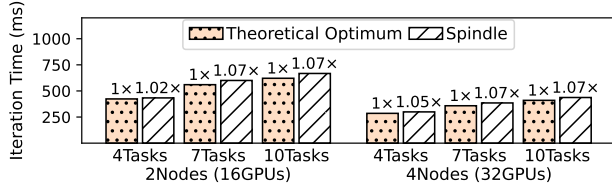


Figure 11. Optimality analysis of Spindle execution planner. Evaluated on Multitask-CLIP.

global optimal parallel plan for multi-tasks. Overall, Spindle’s unified optimization captures a close-to-optimal execution plan with workload balance, leading to consistently high utilization in all aspects.

5.4 Time Breakdown

Fig. 10 shows the runtime breakdown for Spindle and DeepSpeed across various workloads, primarily consisting of forward and backward propagation, parameter synchronization, and inter-wave *send* and *receive*. We’ve isolated parameter synchronization from the backward phase for individual analysis. In MT MM training, the forward and backward propagation dominates the runtime, typically accounting for 80%-95% due to the large number of tasks and computational demands. Spindle focuses on reducing this significant time component through flexible resource allocation and scheduling. Parameter synchronization usually consumes a small fraction of the time, about 5%-15%, since it only occurs after accumulating gradients from multiple tasks. Furthermore, while Spindle introduces extra overhead for inter-wave *send* and *receive*, this overhead remains minimal, typically not exceeding 6%, thanks to the device placement mechanism that avoids unnecessary communications.

Ablation on Device Placement. We conduct an ablation study on the device placement strategy in §3.5, focusing on its impact on inter-wave communication overhead, which is the extra overhead of our system. Specifically, we compare it with a sequential placement strategy, which assigns each

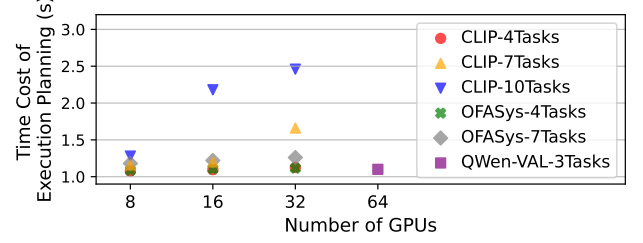


Figure 12. Time cost (s) of Spindle’s execution planner.

MetaOp with consecutive devices. In Fig. 10, our results indicate that the inter-wave communication overhead of the sequential placement strategy is approximately 3-6 times greater than that of Spindle, taking up to 27% of the end-to-end training time. However, Spindle’s placement strategies reduces this overhead to 6%. This demonstrates the effectiveness of our locality-aware placement, which significantly reduces the extra communication overhead.

5.5 Execution Planner Evaluation

Optimality Analysis. We analyze the optimality of Spindle execution planner in Fig. 11 by comparing the iteration time to the theoretical optimum $\tilde{C}^*$ derived from Theorem 1. Although unachievable due to the relaxed constraints (§3.3), $\tilde{C}^*$ serves as a theoretical performance upper bound. The Spindle execution planner preserves most of the optimum property when finding the practical solution (e.g., Cond. (10a) (10b)), but still introducing minor biases (e.g., reinstating l ’s to integers in §3.3, resource extension in §3.4). In Fig. 11, we find that across various configurations, the deviation between Spindle and theoretical optimum is consistently low, below 7%. This observation underscores the effectiveness of Spindle in offering a practical and near-optimal execution plan for MT MM models.

Complexity Analysis. We briefly analyze the complexity of execution planner. Given MetaOps’ scalability curves, the planning process consists of three parts: resource allocation, wavefront scheduling, and device placement. Among the first two parts, most of the time is spent on solving the continuous optimization problem via bisection search (Appendix B [2]). In comparison, the complexity of wavefront scheduling is relatively small, scaling linearly with the number of waves, which is at most twice the number of MetaOps. This is because each wave consumes all layers of at least one ASL-tuple (§3.4 ③), while each MetaOp produces two ASL-tuples (§3.3 bi-point discretized allocation). The third part, device placement, uses a constrained-depth recursive search with simple heuristics. The searching time may vary, but is generally within an acceptable range. As shown in Fig. 12, Spindle effectively generates the execution plans within 3 seconds across all experiments. Moreover, the plan will only

be regenerated when input data workload changes, which is not very often compared to the overall training process.

More Experimental Results. Due to the space constraint, we put more experimental results and analysis in the Appendix [2], including more details of experimental workloads in Appendix C, evaluation of dynamicity performance in Appendix D, larger-scale simulations in Appendix E, comparison on single-task multi-modal (STMM) workloads in Appendix F, memory consumption analysis in Appendix G, and system implementation performance evaluation in Appendix H. Please kindly refer to our Appendix for more details.

6 Related Works

Cluster Scheduling for DL Jobs. GPU clusters often design cluster schedulers to coordinate resource allocation and the execution order among multiple DL jobs. Some cluster schedulers [23, 78] allocate resources to jobs based directly on user-specified requirements. Others [35, 41, 45, 49, 53, 54, 79, 85] automatically allocate resources to each job based on the job scalability to the computing resource, many of them minimizing job completion time (JCT). For instance, Optimus [53] introduces the marginal gain to guide resource allocation, aiming to minimize job completion time. Here, we highlight the difference of these works and MT MM model training. Unlike the independence among jobs in cluster scheduling, MT MM training involves execution dependencies among tasks. Furthermore, while traditional scheduling focuses on job-level allocation, MT MM training requires finer-grained strategies to address intra-task workload heterogeneity.

Training Optimization on Heterogeneous Cluster. This line of research focuses on optimizing the distributed training efficiency of DL models on heterogeneous GPU clusters [30, 32, 42, 43, 52, 65]. While these works primarily concentrate on optimizing single model training and address hardware heterogeneity, Spindle mainly focus on more complex MT MM models, and addresses the challenges posed by the workload heterogeneity of MT MM models.

7 Conclusion

Efficient training of MT MM models faces significant system challenges due to the workload heterogeneity and complex execution dependency. In this paper, we proposed Spindle to facilitate efficient training of MT MM models via wavefront scheduling, which jointly optimizes heterogeneity-aware workload parallelization and dependency-driven execution scheduling. Extensive experiments demonstrate the consistent superior performance of Spindle, outperforming existing SOTA training systems with speedup ratio up to 71%.

Acknowledgments

This work is supported by National Science and Technology Major Project (2022ZD0116315), National Natural Science Foundation of China (U23B2048, 62402011), Beijing Municipal Science and Technology Project (Z231100010323002), China National Postdoctoral Program for Innovative Talents (BX20230012), China Postdoctoral Science Foundation (2024M750103), Beijing Natural Science Foundation (4244080), research grant No. IPT-2024JK29, Alibaba-PKU joint program, and High-performance Computing Platform of Peking University. Fangcheng Fu, Xupeng Miao and Bin Cui are the corresponding authors.

References

- [1] 2024. Introducing Meta Llama 3: The most capable openly available LLM to date. <https://ai.meta.com/blog/meta-llama-3/>.
- [2] 2025. Spindle Appendix. <https://github.com/AFDWang/ASPLOS25-Spindle-Supplemental-Material/blob/main/Appendix.pdf>.
- [3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [4] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob L. Menick, Sebastian Borgeaud, Andy Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karén Simonyan. 2022. Flamingo: a Visual Language Model for Few-Shot Learning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/960a172bc7fbf0177cccb411a7d800-Abstract-Conference.html
- [5] Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Slav Petrov, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy P. Lillicrap, Angeliki Lazaridou, Orhan Firat, James Molloy, Michael Isard, Paul Ronald Barham, Tom Hennigan, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, Ryan Doherty, Eli Collins, Clemens Meyer, Eliza Rutherford, Erica Moreira, Kareem Ayoub, Megha Goel, George Tucker, Enrique Piqueras, Maxim Krikun, Iain Barr, Nikolay Savinov, Ivo Danihelka, Becca Roelofs, Anaïs White, Anders Andreassen, Tamara von Glehn, Lakshman Yagati, Mehran Kazemi, Lucas Gonzalez, Misha Khalman, Jakub Sygnowski, and et al. 2023. Gemini: A Family of Highly Capable Multimodal Models. *CoRR abs/2312.11805* (2023). doi:10.48550/ARXIV.2312.11805 arXiv:2312.11805
- [6] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lucic, and Cordelia Schmid. 2021. ViViT: A Video Vision Transformer. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*. IEEE, 6816–6826. doi:10.1109/ICCV48922.2021.00676
- [7] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. 2020. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo

- Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). <https://proceedings.neurips.cc/paper/2020/hash/92d1e1eb1cd6f9fba3227870bb6d7f07-Abstract.html>
- [8] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Frontier Large Vision-Language Model with Versatile Abilities. *CoRR* abs/2308.12966 (2023). doi:10.48550/ARXIV.2308.12966 arXiv:2308.12966
- [9] Jinze Bai, Rui Men, Hao Yang, Xuancheng Ren, Kai Dang, Yichang Zhang, Xiaohuan Zhou, Peng Wang, Sinan Tan, An Yang, Zeyu Cui, Yu Han, Shuai Bai, Wenbin Ge, Jianxin Ma, Junyang Lin, Jingren Zhou, and Chang Zhou. 2022. OFASys: A Multi-Modal Multi-Task Learning System for Building Generalist Models. *CoRR* abs/2212.04408 (2022). doi:10.48550/ARXIV.2212.04408 arXiv:2212.04408
- [10] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. 2022. BEiT: BERT Pre-Training of Image Transformers. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022*. OpenReview.net. <https://openreview.net/forum?id=p-BhZS59o4>
- [11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.
- [12] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. *CoRR* abs/1604.06174 (2016). arXiv:1604.06174 <http://arxiv.org/abs/1604.06174>
- [13] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, Ji Ma, Jiaqi Wang, Xiaoyi Dong, Hang Yan, Hewei Guo, Conghui He, Botian Shi, Zhenjiang Jin, Chao Xu, Bin Wang, Xingjian Wei, Wei Li, Wenjian Zhang, Bo Zhang, Pinlong Cai, Licheng Wen, Xiangchao Yan, Min Dou, Lewei Lu, Xizhou Zhu, Tong Lu, Dahua Lin, Yu Qiao, Jifeng Dai, and Wenhao Wang. 2024. How far are we to GPT-4V? Closing the gap to commercial multimodal models with open-source suites. *Sci. China Inf. Sci.* 67, 220101 (2024). doi:10.1007/s11432-024-4231-5
- [14] Yunfei Chu, Jin Xu, Xiaohuan Zhou, Qian Yang, Shiliang Zhang, Zhijie Yan, Chang Zhou, and Jingren Zhou. 2023. Qwen-Audio: Advancing Universal Audio Understanding via Unified Large-Scale Audio-Language Models. *CoRR* abs/2311.07919 (2023). doi:10.48550/ARXIV.2311.07919 arXiv:2311.07919
- [15] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *CoRR* abs/2307.08691 (2023). doi:10.48550/ARXIV.2307.08691 arXiv:2307.08691
- [16] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html
- [17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT*. 4171–4186.
- [18] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *ICLR*.
- [19] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. 2023. PaLM-E: An Embodied Multimodal Language Model. In *International Conference on Machine Learning, ICML 2023, 23–29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 8469–8488. <https://proceedings.mlr.press/v202/driess23a.html>
- [20] Maciej Drozdowski. 2009. *Scheduling for Parallel Processing* (1st ed.). Springer Publishing Company, Incorporated.
- [21] Hao Feng, Qi Liu, Hao Liu, Jingqun Tang, Wengang Zhou, Houqiang Li, and Can Huang. 2024. DocPedia: unleashing the power of large multimodal model in the frequency domain for versatile document understanding. *Sci. China Inf. Sci.* 67, 220106 (2024). doi:10.1007/s11432-024-4250-y
- [22] Rohit Girdhar, Alaaeldin El-Nouby, Zhuang Liu, Mannat Singh, Kalyan Vasudev Alwala, Armand Joulin, and Ishan Misra. 2023. ImageBind One Embedding Space to Bind Them All. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17–24, 2023*. IEEE, 15180–15190. doi:10.1109/CVPR52729.2023.01457
- [23] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Harry Liu, and Chuanxiong Guo. 2019. Tiresias: A GPU Cluster Manager for Distributed Deep Learning. In *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26–28, 2019*, Jay R. Lorch and Minlan Yu (Eds.). USENIX Association, 485–500. <https://www.usenix.org/conference/nsdi19/presentation/gu>
- [24] Lei Guan, Dong-Sheng Li, Jiye Liang, Wen-Jian Wang, Ke-shi Ge, and Xicheng Lu. 2024. Advances of Pipeline Model Parallelism for Deep Learning Training: An Overview. *J. Comput. Sci. Technol.* 39, 3 (2024), 567–584. doi:10.1007/S11390-024-3872-3
- [25] Andrey Guzhov, Federico Raue, Jörn Hees, and Andreas Dengel. 2022. Audioclip: Extending Clip to Image, Text and Audio. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2022, Virtual and Singapore, 23–27 May 2022*. IEEE, 976–980. doi:10.1109/ICASSP43922.2022.9747631
- [26] Roger W. Hockney. 1994. The Communication Challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Comput.* 20, 3 (1994), 389–398. doi:10.1016/S0167-8191(06)80021-9
- [27] Jun Huang, Zhen Zhang, Shuai Zheng, Feng Qin, and Yida Wang. 2024. {DISTMM}: Accelerating Distributed Multimodal Model Training. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1157–1171.
- [28] Yanping Huang, Youlong Cheng, Ankur Bapna, et al. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *NeurIPS*.
- [29] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc V. Le, Yun-Hsuan Sung, Zhen Li, and Tom Duerig. 2021. Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18–24 July 2021, Virtual Event (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 4904–4916. <http://proceedings.mlr.press/v139/jia21b.html>
- [30] Xianyan Jia, Le Jiang, Ang Wang, Wencong Xiao, Ziji Shi, Jie Zhang, Xinyuan Li, Langshi Chen, Yong Li, Zhen Zheng, et al. 2022. Whale: Efficient giant model training over heterogeneous {GPUs}. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 673–688.
- [31] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond Data and Model Parallelism for Deep Neural Networks. In *MLSys*.
- [32] Dacheng Li, Hongyi Wang, Eric P. Xing, and Hao Zhang. 2022. AMP: Automatically Finding Model Parallel Strategies with Heterogeneity Awareness. In *Advances in Neural Information Processing Systems*

- 35: *Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/2b4bfa1cebe78d125fef7ea6ffcf6d-Abstract-Conference.html
- [33] Junnan Li, Dongxu Li, Silvio Savarese, and Steven C. H. Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 19730–19742. <https://proceedings.mlr.press/v202/li23q.html>
- [34] Junnan Li, Dongxu Li, Caoming Xiong, and Steven C. H. Hoi. 2022. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (Eds.). PMLR, 12888–12900. <https://proceedings.mlr.press/v162/li22n.html>
- [35] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. 2023. Lyra: Elastic scheduling for deep learning clusters. In *Proceedings of the Eighteenth European Conference on Computer Systems*. 835–850.
- [36] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proc. VLDB Endow.* 13, 12 (2020), 3005–3018. doi:10.14778/3415478.3415530
- [37] Dongyang Liu, Renrui Zhang, Longtian Qiu, Siyuan Huang, Weifeng Lin, Shitian Zhao, Shijie Geng, Ziyi Lin, Peng Jin, Kaipeng Zhang, Wenqi Shao, Chao Xu, Conghui He, Junjun He, Hao Shao, Pan Lu, Yu Qiao, Hongsheng Li, and Peng Gao. 2024. SPHINX-X: scaling data and parameters for a family of multi-modal large language models. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 1314, 21 pages.
- [38] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html
- [39] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *ICCV*. IEEE, 9992–10002.
- [40] Jiasen Lu, Christopher Clark, Rowan Zellers, Roozbeh Mottaghi, and Aniruddha Kembhavi. 2023. UNIFIED-IO: A Unified Model for Vision, Language, and Multi-modal Tasks. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/pdf?id=E01k9048soZ>
- [41] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. 2020. Themis: Fair and Efficient GPU Cluster Scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*, Ranjita Bhagwan and George Porter (Eds.). USENIX Association, 289–304. <https://www.usenix.org/conference/nsdi20/presentation/mahajan>
- [42] Xupeng Miao, Xiaonan Nie, Yingxia Shao, Zhi Yang, Jiawei Jiang, Lingxiao Ma, and Bin Cui. 2021. Heterogeneity-Aware Distributed Machine Learning Training via Partial Reduce. In *SIGMOD*. ACM, 2262–2270.
- [43] Xupeng Miao, Yining Shi, Zhi Yang, Bin Cui, and Zhihao Jia. 2023. SDPipe: A Semi-Decentralized Framework for Heterogeneity-aware Pipeline-parallel Training. *Proc. VLDB Endow.* 16, 9 (2023), 2354–2363. doi:10.14778/3598581.3598604
- [44] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proc. VLDB Endow.* 16, 3 (2022), 470–479. doi:10.14778/3570690.3570697
- [45] Zizhao Mo, Huanle Xu, and Chengzhong Xu. 2024. Heet: Accelerating Elastic Training in Heterogeneous Deep Learning Clusters. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 499–513.
- [46] Seungwhan Moon, Andrea Madotto, Zhaojiang Lin, Tushar Nagarajan, Matt Smith, Shashank Jain, Chun-Fu Yeh, Prakash Murugesan, Peyman Heidari, Yue Liu, Kavya Srinet, Babak Damavandi, and Anuj Kumar. 2023. AnyMAL: An Efficient and Scalable Any-Modality Augmented Language Model. *CoRR abs/2309.16058* (2023). doi:10.48550/ARXIV.2309.16058 arXiv:2309.16058
- [47] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: generalized pipeline parallelism for DNN training. In *SOSP*. 1–15.
- [48] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-efficient pipeline-parallel dnn training. In *International Conference on Machine Learning*. PMLR, 7937–7947.
- [49] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. 2020. Heterogeneity-Aware Cluster Scheduling Policies for Deep Learning Workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 481–498. <https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak>
- [50] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, et al. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. In *SC*. ACM, 58:1–58:15.
- [51] OpenAI. 2023. ChatGPT. <https://chat.openai.com/chat>.
- [52] Jay H. Park, Gyeongchan Yun, Chang M. Yi, Nguyen T. Nguyen, Seungmin Lee, Jaesik Choi, Sam H. Noh, and Young-ri Choi. 2020. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters through Integration of Pipelined Model Parallelism and Data Parallelism. In *2020 USENIX Annual Technical Conference, USENIX ATC 2020, July 15-17, 2020, Ada Gavrilovska and Erez Zadok (Eds.)*. USENIX Association, 307–321. <https://www.usenix.org/conference/atc20/presentation/park>
- [53] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. 2018. Optimus: an efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, Porto, Portugal, April 23-26, 2018*, Rui Oliveira, Pascal Felber, and Y. Charlie Hu (Eds.). ACM, 3:1–3:14. doi:10.1145/3190508.3190517
- [54] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. 2021. Pollux: Co-adaptive Cluster Scheduling for Goodput-Optimized Deep Learning. In *15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2021, July 14-16, 2021*, Angela Demke Brown and Jay R. Lorch (Eds.). USENIX Association. <https://www.usenix.org/conference/osdi21/presentation/qiao>
- [55] Xueyang Qin, Lishuang Li, Jingyao Tang, Fei Hao, Meiling Ge, and Guangyao Pang. 2024. Multi-Task Visual Semantic Embedding Network for Image-Text Retrieval. *J. Comput. Sci. Technol.* 39, 4 (2024), 811–826. doi:10.1007/S11390-024-4125-1
- [56] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *ICML*, Vol. 139. PMLR, 8748–8763.

- [57] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust Speech Recognition via Large-Scale Weak Supervision. In *International Conference on Machine Learning, ICML 2023, 23–29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 28492–28518. <https://proceedings.mlr.press/v202/radford23a.html>
- [58] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [59] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [60] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR* (2020).
- [61] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: memory optimizations toward training trillion parameter models. In *SC. IEEE/ACM*.
- [62] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *SIGKDD*. 3505–3506.
- [63] Scott E. Reed, Konrad Zolna, Emilio Parisotto, Sergio Gómez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. 2022. A Generalist Agent. *Trans. Mach. Learn. Res.* 2022 (2022). <https://openreview.net/forum?id=1ikK0kHjvj>
- [64] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14–16, 2021, Irina Calciu and Geoff Kuenning (Eds.)*. USENIX Association, 551–564. <https://www.usenix.org/conference/atc21/presentation/ren-jie>
- [65] Linghao Song, Fan Chen, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. 2020. AccPar: Tensor Partitioning for Heterogeneous Deep Learning Accelerators. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2020, San Diego, CA, USA, February 22–26, 2020*. IEEE, 342–355. doi:10.1109/HPCA47549.2020.00036
- [66] Zhan Tong, Yibing Song, Jue Wang, and Limin Wang. 2022. VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 – December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/416f9cb3276121c42eebb86352a4354a-Abstract-Conference.html
- [67] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR abs/2302.13971* (2023). doi:10.48550/ARXIV.2302.13971 arXiv:2302.13971
- [68] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutu Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Illyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR abs/2307.09288* (2023). doi:10.48550/ARXIV.2307.09288 arXiv:2307.09288
- [69] Colin Unger, Zhihao Jia, Wei Wu, et al. 2022. Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization. In *OSDI*. 267–284.
- [70] Changhan Wang, Anne Wu, Juan Pino, Alexei Baevski, Michael Auli, and Alexis Conneau. 2021. Large-Scale Self- and Semi-Supervised Learning for Speech Translation. In *Interspeech 2021, 22nd Annual Conference of the International Speech Communication Association, Brno, Czechia, 30 August – 3 September 2021*, Hynek Hermansky, Honza Cernocký, Lukás Burget, Lori Lamel, Odette Scharenborg, and Petr Motlíček (Eds.). ISCA, 2242–2246. doi:10.21437/INTERSPEECH.2021-1912
- [71] Guozheng Wang, Yongmei Lei, Zeyu Zhang, and Cunlu Peng. 2023. A Communication Efficient ADMM-based Distributed Algorithm Using Two-Dimensional Torus Grouping AllReduce. *Data Sci. Eng.* 8, 1 (2023), 61–72. doi:10.1007/S41019-022-00202-7
- [72] Guanhua Wang, Heyang Qin, Sam Ade Jacobs, Connor Holmes, Samyam Rajbhandari, Olatunji Ruwase, Feng Yan, Lei Yang, and Yuxiong He. 2023. ZeRO++: Extremely Efficient Collective Communication for Giant Model Training. *CoRR abs/2306.10209* (2023). doi:10.48550/ARXIV.2306.10209 arXiv:2306.10209
- [73] Peng Wang, An Yang, Rui Men, Junyang Lin, Shuai Bai, Zhikang Li, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022. OFA: Unifying Architectures, Tasks, and Modalities Through a Simple Sequence-to-Sequence Learning Framework. In *International Conference on Machine Learning, ICML 2022, 17–23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (Eds.). PMLR, 23318–23340. <https://proceedings.mlr.press/v162/wang22al.html>
- [74] Wenhui Wang, Hangbo Bao, Li Dong, Johan Björck, Zhiliang Peng, Qiang Liu, Kriti Aggarwal, Owais Khan Mohammed, Saksham Singhal, Subhojit Som, and Furu Wei. 2023. Image as a Foreign Language: BEIT Pretraining for Vision and Vision-Language Tasks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17–24, 2023*. IEEE, 19175–19186. doi:10.1109/CVPR52729.2023.01838
- [75] Yujie Wang, Youhe Jiang, Xupeng Miao, Fangcheng Fu, Shenhan Zhu, Xiaonan Nie, Yaofeng Tu, and Bin Cui. 2024. Improving Automatic Parallel Training via Balanced Memory Workload Optimization. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [76] Jan Weglarz. 1981. Project Scheduling with Continuously-Divisible, Doubly Constrained Resources. *Manage. Sci.* 27, 9 (sep 1981), 1040–1053. doi:10.1287/mnsc.27.9.1040
- [77] Jan Weglarz. 1982. Modelling and control of dynamic resource allocation project scheduling systems. *Optimization and Control of Dynamic Operational Research Models* (1982), 105–140.
- [78] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. 2018. Gandiva: Introspective Cluster Scheduling for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8–10, 2018*, Andrea C.

- Arpaci-Dusseau and Geoff Voelker (Eds.). USENIX Association, 595–610. <https://www.usenix.org/conference/osdi18/presentation/xiao>
- [79] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. 2020. AntMan: Dynamic Scaling on GPU Clusters for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4–6, 2020*. USENIX Association, 533–548. <https://www.usenix.org/conference/osdi20/presentation/xiao>
- [80] Hu Xu, Gargi Ghosh, Po-Yao Huang, Dmytro Okhonko, Armen Aghajanyan, Florian Metze, Luke Zettlemoyer, and Christoph Feichtenhofer. 2021. VideoCLIP: Contrastive Pre-training for Zero-shot Video-Text Understanding. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7–11 November, 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, 6787–6800. doi:10.18653/V1/2021.EMNLP-MAIN.544
- [81] Mengying Xu, Linyin Luo, Hanjiang Lai, and Jian Yin. 2024. Category-Level Contrastive Learning for Unsupervised Hashing in Cross-Modal Retrieval. *Data Sci. Eng.* 9, 3 (2024), 251–263. doi:10.1007/S41019-024-00248-9
- [82] Zhewei Yao, Xiaoxia Wu, Conglong Li, Minjia Zhang, Heyang Qin, Olatunji Ruwase, Ammar Ahmad Awan, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed-VisualChat: Multi-Round Multi-Image Interleave Chat via Multi-Modal Causal Attention. *arXiv preprint arXiv:2309.14327* (2023).
- [83] Jiahui Yu, Zirui Wang, Vijay Vasudevan, Legg Yeung, Mojtaba Seyedhosseini, and Yonghui Wu. 2022. CoCa: Contrastive Captioners are Image-Text Foundation Models. *Trans. Mach. Learn. Res.* 2022 (2022). <https://openreview.net/forum?id=Ee277P3AYC>
- [84] Lu Yuan, Dongdong Chen, Yi-Ling Chen, Noel Codella, Xiyang Dai, Jianfeng Gao, Houdong Hu, Xuedong Huang, Boxin Li, Chunyuan Li, Ce Liu, Mengchen Liu, Zicheng Liu, Yumao Lu, Yu Shi, Lijuan Wang, Jianfeng Wang, Bin Xiao, Zhen Xiao, Jianwei Yang, Michael Zeng, Luowei Zhou, and Pengchuan Zhang. 2021. Florence: A New Foundation Model for Computer Vision. *CoRR* abs/2111.11432 (2021). arXiv:2111.11432 <https://arxiv.org/abs/2111.11432>
- [85] Haoyu Zhang, Logan Stafman, Andrew Or, and Michael J. Freedman. 2017. SLAQ: quality-driven scheduling for distributed machine learning. In *Proceedings of the 2017 Symposium on Cloud Computing, SoCC 2017, Santa Clara, CA, USA, September 24–27, 2017*. ACM, 390–404. doi:10.1145/3127479.3127490
- [86] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Mylène Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proc. VLDB Endow.* 16, 12 (2023), 3848–3860. doi:10.14778/3611540.3611569
- [87] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11–13, 2022*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, 559–578. <https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>
- [88] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023. MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models. *CoRR* abs/2304.10592 (2023). doi:10.48550/ARXIV.2304.10592 arXiv:2304.10592